

TRELLIS.2 API

HTTP Interface Guide for AI Agents & Blender Addons

Host: `http://styleengine2:8195`

Overview

The TRELLIS.2 API is a single-worker HTTP server. All requests are multipart/form-data (not JSON). Jobs are queued and processed one at a time. The workflow is always: submit a job → poll status → download result.

IMPORTANT All endpoints accept multipart/form-data, NOT application/json. File uploads and parameters are sent in the same form body.

Endpoints

Method	Path	Description
POST	<code>/generate</code>	Submit Image → 3D job. Returns <code>job_id</code> .
POST	<code>/retexture</code>	Submit Mesh + Image → retexture job. Returns <code>job_id</code> .
GET	<code>/status/{job_id}</code>	Poll job status, progress, and message.
GET	<code>/download/{job_id}/{index}</code>	Download completed GLB file (index=0 for first output).
GET	<code>/jobs</code>	List last 20 jobs with their status.
DELETE	<code>/jobs/{job_id}</code>	Delete a job and its files from disk.
GET	<code>/health</code>	Server health, VRAM usage, queue depth.

POST `/generate`

Submit an image to generate a 3D mesh. The image must be RGBA PNG with a transparent background (alpha channel used directly — rembg is disabled).

Request

Content-Type: `multipart/form-data` — file + parameters in one body

Required Fields

Field	Type	Default	Description
<code>image</code>	file (PNG)	required	RGBA PNG with transparent background. Alpha channel used directly.

Generation Parameters

Field	Type	Default	Description
pipeline_type	string	1024_cascade	Resolution mode: "512" "1024" "1024_cascade" "1536_cascade"
seed	int	42	Random seed for reproducibility. Range: 0–99999
steps	int	12	Diffusion steps per stage (sparse, shape, texture). Range: 4–50. Higher = more detail, slower.
guidance	float	7.5	CFG guidance strength. Range: 1–20. Higher = closer to image, less creative.
num_samples	int	1	Number of mesh variants to generate. Range: 1–4. Each is downloadable separately.
max_num_tokens	int	49152	Token budget for sparse structure. Values: 16384 32768 49152 65536. Reduce if CUDA OOM.

Mesh Export Parameters

Field	Type	Default	Description
texture_size	int	4096	Output texture atlas resolution in pixels. Values: 1024 2048 4096.
decimation_target	int	1000000	Target vertex count after decimation. Range: 10000–10000000.
remesh	bool	true	Run dual contouring remesh for cleaner topology. Send as string: "true" or "false".
remesh_band	float	1.0	Band width for remeshing. Range: 0.5–3.0. Higher = smoother but less detail.
remesh_project	float	0.0	How much to project vertices back onto original surface. Range: 0–1.

pipeline_type Reference

Value	Resolution	A100 time	Notes
512	512³	~3s	Fast prototyping. Low detail.
1024	1024³	~17s	Standard quality.
1024_cascade	1024³	~17s	DEFAULT. Cascade sampler, better coherence.
1536_cascade	1536³	~60s	Max quality. More prone to hallucination on complex shapes.

Response

Field	Type	Description
job_id	string	UUID identifying this job. Store this to poll status and download.
queue_position	int	Position in queue. 0 = processing now, 1 = next, etc.

Example Request (Python / urllib)

```
import urllib.request, json

boundary = "----BlenderTRELLIS"

with open("/tmp/render.png", "rb") as f:
    img_data = f.read()

def field(name, value):
    return (
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="{name}"\r\n\r\n'
        f'{value}\r\n'
    ).encode()

body = f"--{boundary}\r\n".encode()
body += b'Content-Disposition: form-data; name="image"; filename="input.png"\r\n'
body += b"Content-Type: image/png\r\n\r\n"
body += img_data + b"\r\n"
body += field("pipeline_type", "1024_cascade")
body += field("seed", 42)
body += field("steps", 12)
body += field("guidance", 7.5)
body += field("num_samples", 1)
body += field("max_num_tokens", 49152)
body += field("texture_size", 4096)
body += field("decimation_target", 1000000)
body += field("remesh", "true")
body += field("remesh_band", 1.0)
body += field("remesh_project", 0.0)
body += f"--{boundary}--\r\n".encode()

req = urllib.request.Request(
    "http://styleengine2:8195/generate",
    data=body,
    headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
    method="POST"
)
result = json.loads(urllib.request.urlopen(req).read())
# result = {"job_id": "abc123...", "queue_position": 0}
```

POST /retexture

Submit an existing mesh + reference image to retexture it. The mesh geometry is unchanged — only the texture is regenerated using the reference image as guidance.

Required Fields

Field	Type	Default	Description
image	file (PNG)	required	RGBA PNG reference image. Alpha channel used directly.
mesh	file	required	Mesh file to retexture. Accepted: .glb .ply .obj .gltf

Texturing Parameters

Field	Type	Default	Description
seed	int	0	Random seed. Range: 0–99999.
tex_resolution	int	1024	Voxel resolution for texturing pass. Values: 512 1024 1536.
texture_size	int	2048	Output texture atlas resolution. Values: 1024 2048 4096.
tex_steps	int	12	Diffusion steps. Range: 1–50.
tex_guidance	float	1.0	CFG guidance strength. Range: 0.1–10. Lower = more creative, higher = closer to reference.
tex_guidance_rescale	float	0.0	Guidance rescale factor. Range: 0–1. 0 = off. Reduces oversaturation at high guidance.
tex_rescale_t	float	3.0	Rescale T controls interpolation aggressiveness. Range: 1–6. Lower = smoother, higher = sharper.

Example Request (Python / urllib)

```
boundary = "----BlenderTRELLIS"

with open("/tmp/reference.png", "rb") as f: img_data = f.read()
with open("/tmp/mesh.glb", "rb") as f: mesh_data = f.read()

body = f"--{boundary}\r\n".encode()
body += b'Content-Disposition: form-data; name="image"; filename="input.png"\r\n'
body += b'Content-Type: image/png\r\n\r\n'
body += img_data + b"\r\n"
body += f"--{boundary}\r\n".encode()
body += b'Content-Disposition: form-data; name="mesh"; filename="mesh.glb"\r\n'
body += b'Content-Type: model/gltf-binary\r\n\r\n'
body += mesh_data + b"\r\n"
body += field("seed", 0)
body += field("tex_resolution", 1024)
body += field("texture_size", 2048)
body += field("tex_steps", 12)
body += field("tex_guidance", 1.0)
body += field("tex_guidance_rescale", 0.0)
body += field("tex_rescale_t", 3.0)
body += f"--{boundary}--\r\n".encode()

req = urllib.request.Request(
    "http://styleengine2:8195/retexture",
    data=body,
```

```

headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
method="POST"
)
result = json.loads(urllib.request.urlopen(req).read())

```

GET /status/{job_id}

Poll this endpoint to track job progress. Call every 1–2 seconds while status is "queued" or "processing".

Response Fields

Field	Type	Description
job_id	string	The job UUID.
status	string	"queued" "processing" "done" "failed"
progress	float	0.0 to 1.0. Multiply by 100 for percentage.
message	string	Human-readable current step description.
elapsed	float null	Seconds since job started processing. Null if still queued.
gen_time	float null	Total generation time in seconds. Only set when done.
num_outputs	int	Number of GLB files ready to download. Equals num_samples when done.
queue_pos	int	Queue position at submission time (informational).
mode	string	"generate" or "retexture" — which pipeline produced this job.
error	string null	Error message if status is "failed". Null otherwise.

Status Flow

```

queued → processing → done
                → failed

```

NOTE

A job stays in "queued" until the previous job finishes. The server processes one job at a time. queue_pos tells you how many jobs are ahead.

Example Polling Loop (Python)

```

import urllib.request, json, time

def poll_until_done(job_id, host="http://styleengine2:8195", interval=2.0):
    while True:
        r = urllib.request.urlopen(f"{host}/status/{job_id}")
        s = json.loads(r.read())
        print(f"{s['status']} {s['progress']*100:.0f}% - {s['message']}")
        if s["status"] == "done":
            return s["num_outputs"]
        if s["status"] == "failed":

```

```
raise RuntimeError(s["error"])
time.sleep(interval)
```

GET /download/{job_id}/{index}

Download a completed GLB file. Index is 0-based and corresponds to num_samples (e.g. if you requested num_samples=3, download index 0, 1, and 2).

Example

```
import urllib.request

def download_glb(job_id, output_path, index=0, host="http://styleengine2:8195"):
    url = f"{host}/download/{job_id}/{index}"
    urllib.request.urlretrieve(url, output_path)
    print(f"Saved to {output_path}")

# For num_samples=2:
download_glb(job_id, "/tmp/output_0.glb", index=0)
download_glb(job_id, "/tmp/output_1.glb", index=1)
```

RETURNS

Binary GLB file (model/gltf-binary). Content-Disposition header includes a suggested filename: trellis2_{mode}_{job_id[:8]}_{index}.glb

GET /health

Check server readiness before submitting jobs. Returns VRAM usage and pipeline load status.

Response Fields

Field	Type	Description
pipeline_loaded	bool	True if the Image→3D pipeline is loaded and ready.
tex_pipeline_loaded	bool	True if the Texturing pipeline is loaded and ready.
queue_depth	int	Number of jobs currently waiting in the queue.
total_jobs	int	Total jobs in memory since last server start.
gpu	string	GPU model name (e.g. "NVIDIA A100-SXM4-40GB").
vram_used_gb	float	VRAM currently allocated in GB.
vram_total_gb	float	Total VRAM available in GB.
vram_free_gb	float	Free VRAM in GB (total minus used).

Example

```

import urllib.request, json

health = json.loads(urllib.request.urlopen("http://styleengine2:8195/health").read())

if not health["pipeline_loaded"]:
    print("Pipeline still loading, wait ~2 minutes after service start")
elif health["queue_depth"] > 3:
    print(f"Queue is busy ({health['queue_depth']} jobs waiting)")
else:
    print(f"Ready. VRAM: {health['vram_used_gb']}GB / {health['vram_total_gb']}GB")

```

Full Workflow Example

This is the complete recommended pattern for Blender addons and AI agents. Uses only Python stdlib (no requests library needed).

```

import urllib.request, json, time

HOST = "http://styleengine2:8195"
boundary = "----BlenderTRELIS"

def field(name, value):
    return (
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="{name}"\r\n\r\n'
        f"{value}\r\n"
    ).encode()

# — Step 1: Check server is ready —————
health = json.loads(urllib.request.urlopen(f"{HOST}/health").read())
assert health["pipeline_loaded"], "Pipeline not ready yet"

# — Step 2: Submit job —————
with open("/tmp/myobject.png", "rb") as f:
    img_data = f.read()

body = f"--{boundary}\r\n".encode()
body += b'Content-Disposition: form-data; name="image"; filename="input.png"\r\n'
body += b'Content-Type: image/png\r\n\r\n'
body += img_data + b"\r\n"
body += field("pipeline_type", "1024_cascade")
body += field("seed", 42)
body += field("steps", 20)
body += field("guidance", 7.5)
body += field("num_samples", 1)
body += field("texture_size", 4096)
body += field("remesh", "true")
body += f"--{boundary}--\r\n".encode()

req = urllib.request.Request(
    f"{HOST}/generate", data=body,
    headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
    method="POST"
)
result = json.loads(urllib.request.urlopen(req).read())
job_id = result["job_id"]
print(f"Job submitted: {job_id}")

```

```

# — Step 3: Poll until done —————
while True:
    s = json.loads(urllib.request.urlopen(f"{HOST}/status/{job_id}").read())
    print(f"[{s['progress']*100:.0f}%] {s['message']}")
    if s["status"] == "done": break
    if s["status"] == "failed": raise RuntimeError(s["error"])
    time.sleep(2)

# — Step 4: Download GLB(s) —————
for i in range(s["num_outputs"]):
    out = f"/tmp/output_{i}.glb"
    urllib.request.urlretrieve(f"{HOST}/download/{job_id}/{i}", out)
    print(f"Downloaded: {out}")

# — Step 5: (Optional) Clean up job —————
req = urllib.request.Request(f"{HOST}/jobs/{job_id}", method="DELETE")
urllib.request.urlopen(req)

```

Blender Modal Operator Pattern

Blender cannot block the main thread. Use a modal timer operator to poll asynchronously. The submission runs in a background thread, and the modal timer checks status every 2 seconds.

```

import bpy, urllib.request, json, threading, time

HOST = "http://styleengine2:8195"
boundary = "----BlenderTRELLIS"

def field(name, value):
    return (
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="{name}"\r\n\r\n'
        f"{value}\r\n"
    ).encode()

class TRELLIS_OT_Generate(bpy.types.Operator):
    bl_idname = "trellis.generate"
    bl_label = "Generate 3D from Image"

    image_path : bpy.props.StringProperty()
    _job_id = None
    _timer = None
    _error = None

    def execute(self, context):
        self._job_id = None
        self._error = None
        threading.Thread(target=self._submit, daemon=True).start()
        wm = context.window_manager
        self._timer = wm.event_timer_add(2.0, window=context.window)
        wm.modal_handler_add(self)
        return {"RUNNING_MODAL"}

    def _submit(self):
        try:
            with open(self.image_path, "rb") as f: img = f.read()
            body = f"--{boundary}\r\n".encode()

```

```

        body += b'Content-Disposition: form-data; name="image"; '
        body += b'filename="input.png"\r\nContent-Type: image/png\r\n\r\n'
        body += img + b"\r\n"
        body += field("pipeline_type", "1024_cascade")
        body += field("steps", 20)
        body += field("guidance", 7.5)
        body += field("texture_size", 4096)
        body += field("remesh", "true")
        body += f"--{boundary}--\r\n".encode()
        req = urllib.request.Request(
            f"{HOST}/generate", data=body,
            headers={"Content-Type":
                f"multipart/form-data; boundary={boundary}"},
            method="POST"
        )
        r = json.loads(urllib.request.urlopen(req).read())
        self._job_id = r["job_id"]
    except Exception as e:
        self._error = str(e)

def modal(self, context, event):
    if event.type != "TIMER":
        return {"PASS_THROUGH"}

    if self._error:
        self.report({"ERROR"}, self._error)
        return self.cancel(context)

    if not self._job_id:
        context.workspace.status_text_set("TRELLIS2: Submitting...")
        return {"PASS_THROUGH"}

    s = json.loads(urllib.request.urlopen(
        f"{HOST}/status/{self._job_id}").read())

    context.workspace.status_text_set(
        f"TRELLIS2: {s['message']} ({s['progress']*100:.0f}%)")

    if s["status"] == "done":
        glb_path = "/tmp/trellis2_output.glb"
        urllib.request.urlretrieve(
            f"{HOST}/download/{self._job_id}/0", glb_path)
        bpy.ops.import_scene.gltf(filepath=glb_path)
        self.report({"INFO"}, f"Done in {s['gen_time']:.1f}s")
        return self.cancel(context)

    if s["status"] == "failed":
        self.report({"ERROR"}, s["error"])
        return self.cancel(context)

    return {"PASS_THROUGH"}

def cancel(self, context):
    context.window_manager.event_timer_remove(self._timer)
    context.workspace.status_text_set(None)
    return {"CANCELLED"}

```

Error Handling

HTTP Code	Meaning	What to do
503	Pipeline not loaded	Server just started. Wait ~2 min and retry. Check /health first.
400	Invalid parameter	Check parameter values against the allowed ranges/options above.
404	Job not found	Job ID is wrong, or job was deleted. Resubmit.
400	Job not complete	Tried to download before status=done. Keep polling.
500	Internal server error	Server crashed. Check journalctl -u trellis2 for traceback.

JOB FAILED	When status=="failed", the error field contains the Python exception string. This is the same as what you'd see in journalctl -u trellis2.
-------------------	--

Quick Reference

GEN defaults	pipeline_type=1024_cascade seed=42 steps=12 guidance=7.5 num_samples=1 max_num_tokens=49152 texture_size=4096 decimation_target=1000000 remesh=true remesh_band=1.0 remesh_project=0.0
---------------------	--

TEX defaults	seed=0 tex_resolution=1024 texture_size=2048 tex_steps=12 tex_guidance=1.0 tex_guidance_rescale=0.0 tex_rescale_t=3.0
---------------------	---

BOOL fields	remesh must be sent as the string "true" or "false", not Python True/False. All other params can be sent as their natural string representation.
--------------------	--

ALPHA	Both /generate and /retexture skip rembg and use the alpha channel directly. Always provide RGBA PNG with a transparent background.
--------------	---